

DO NOT JUST TRUST THE MODEL. TRACE IT.

ML-Model incident, Claude work,
independent audit, and local recovery

The milk failure began in data promotion: 47 valid milk lines became one gold example, then training silently removed the class. This report traces every data stream, Claude-era detour, independent audit, full-SetFit memory repair, metric, remaining risk, and rollback point.

STATUS	LOCAL CANDIDATE
DEPLOYED	NO
PUSHED	NO
RECOVERY	9776523
ROLLBACK	d3e2360 / a60e43f



A beginner-friendly, evidence-backed reconstruction. The supplied Field Guide PDF was used as the documentation framework and left unchanged.

CONTENTS

Read Parts I-IV for the full story. Parts V-XI explain the current system, risks, rollback, Field Guide audit, safe claims, and next work. Appendices are the evidence map.



PART I - THE PROJECT FROM SCRATCH	3
PART II - HOW THE INCIDENT STARTED	11
PART III - WHAT CLAUDE DID	13
PART IV - WHAT CODEX DID	17
PART V - THE CURRENT ARCHITECTURE	23
PART VI - FLAWS, AUDITS, AND OPEN RISKS	24
PART VII - ROLLBACK AND RECOVERY MAP	26
PART VIII - APPLYING THE 15 FIELD-GUIDE HABITS	27
PART IX - DOCUMENT TRUTH MAP	28
PART X - WHAT YOU CAN SAFELY SAY	29
PART XI - RECOMMENDED NEXT WORK	30
APPENDIX A - CHRONOLOGICAL LEDGER	31
APPENDIX B - GLOSSARY	32
APPENDIX C - PRIMARY EVIDENCE INDEX	33
APPENDIX D - FINAL OWNER CHECKLIST	34

Document control and honesty rules

This is a new project-specific edition inspired by the supplied **AI Collaboration Field Guide**. The supplied PDF in Downloads was read and visually inspected. It was not overwritten. Its core instruction - trace the work instead of blindly approving AI summaries - is applied here through:

- a chronological incident record;
- a decision and evidence ledger;
- explicit separation of historical claims from later verification;
- architecture and execution-flow explanations;
- test and audit evidence;
- a rollback map;
- a plain-language mental model.

This report does not pretend that every historical action has an exact timestamp. Claude's pasted transcript records the order of events, but not a timestamp for each message. Git gives exact timestamps only for saved snapshots. Where the record is incomplete, the report says so.

Evidence strength used in this report

Mark	Meaning	Example
VERIFIED	Reproduced from raw data, code, Git, a saved manifest, or an executable local test	47 raw VENTA DE LECHE rows; 1 reached old gold
HISTORICAL CLAIM	Written by the earlier agent at the time, but not independently sufficient	"20/20 on unseen data" before the wording caveat
CORRECTED	A previous statement later shown to be incomplete or wrong	Frozen embeddings were not required by the Mac
OPEN	Not yet safely answerable from current trusted evidence	Correct labels for seven non-canonical sales rows

What this report does not claim

- It does not claim a production release. There is none.
- It does not claim the frontend is fixed. Frontend behavior is recorded as part of the incident, but is outside this recovery.
- It does not claim the new model understands every unseen income phrase.
- It does not claim all old silver labels are trustworthy.
- It does not claim ONNX or deployment parity. No new export was attempted.
- It does not erase Claude's useful findings. It separates useful work from unsafe conclusions.

PART I - THE PROJECT FROM SCRATCH

1. What this project is trying to do

Antillanca has purchase and sales documents. Each document contains line items such as medicine, fuel, fencing, milk sales, cow sales, and many other farm-related transactions. The project assigns each line item to an accounting category code.

Examples:

Line item	Direction	Expected code	Plain meaning
VENTA DE LECHE	VENTAS	ING-0.1	Milk-sale income

Line item	Direction	Expected code	Plain meaning
VENTA DE VACAS	VENTAS	ING-0.2	Cow-sale income
VENTAS TERNEROS	VENTAS	ING-0.4	Calf-sale income
veterinary medicine	COMPRAS	an EXP-* code	Expense
a known electricity meter	COMPRAS	meter-specific code	Deterministic expense category

The prefix matters:

- **ING-*** means income.
- **EXP-*** means operating expense.
- **ADM-*** means administration.

The system has two connected jobs:

- 1 **Offline labeling/backfill:** classify the existing collection of invoice lines.
- 2 **Ongoing classification service:** classify a new line when the application asks.

The classifier supports the labeling job. It is not the complete business product by itself.

2. The data layers in beginner language

Think of the data as moving through increasingly trusted folders.

Layer	What it means	Trust level
Raw XML	Original invoice documents	Source evidence, not labeled
Processed line items	XML converted into rows	Structured, not necessarily labeled
Silver	AI suggestions and audit ledgers	Candidate labels; mixed trust
Gold	Accepted training examples	Supposed to be trusted training truth
Candidate recovery gold	A separate, versioned proposed training set	Audited local candidate
Model	Learns patterns from training inputs and labels	Only as reliable as data and evaluation
Artifact	Exported package used by the API	Production-form package; recovery not exported

The main raw extraction contains approximately 12,103 line items from 5,195 documents. The old gold source had 1,733 rows before the Claude-era broad promotion. The focused recovery produces 1,835 provenance rows and 1,580 distinct normalized model inputs.

“Provenance rows” preserve where examples came from. “Distinct model inputs” are the unique texts the model actually learns from after equivalent duplicates are collapsed.

2A. The complete data-family map

The project contains several data streams that look similar in CSV form but mean very different things. The easiest way to understand them is to separate **business invoices**, **label definitions**, **candidate labels**, and **training examples**.

```
PRIMARY BUSINESS INVOICES
5,195 XML documents
  5,107 purchase documents (COMPRAS)
    88 sales documents (VENTAS)
      |
      | extraction - no accounting label is added here
      v
12,103 invoice line rows
  11,978 purchase lines
    125 sales lines
```

```

CLIENT-DEFINITION STREAMS -----+
  product mappings                  |
  row examples                      |
  future service descriptions       |
                                   |
CLIENT-FILED EXAMPLE-FOLDER STREAMS +--> ORIGINAL GOLD: 1,733 rows
  folder audits                    |
  corrected folder audits          |
                                   |
AI-CANDIDATE / SILVER STREAM       |
  4,135 candidate-pool rows        |
  1,672 per-category ledger rows   |
  1,076 with audit verdicts        |
  1,012 non-rejected usable verdicts -----+

FOCUSED RECOVERY
original gold 1,733
- 3 purchase rows wrongly labeled income
+ 105 verified raw sales rows not already present
= 1,835 provenance rows
= 1,580 distinct normalized model inputs

```

The 5,195 count refers only to the primary Antillanca COMPRAS/VENTAS document folders: 5,107 purchase XML files plus 88 sales XML files. The filesystem also contains 389 XML copies/examples in separate client example trees. Those examples feed the folder-audit stream and must not be confused with the primary invoice corpus.

2B. Raw invoices and processed lines are not automatically training data

`Data/Raw_Data/dte_96685810_COMPRAS/` and `Data/Raw_Data/dte_96685810_VENTAS/` contain the primary XML documents. Extraction converts them to `Data/processed/line_items.csv`.

Stage	COMPRAS	VENTAS	Total	Has trusted accounting label?
Primary XML documents	5,107	88	5,195	No
Extracted invoice lines	11,978	125	12,103	No

One invoice can contain several lines, so 5,195 documents become 12,103 line items. Extraction gives each row fields such as item name, description, provider, amount, source file, and direction. It does **not** decide the accounting category.

Historically, raw rows were not allowed into gold merely because a keyword looked plausible. The exception used in this recovery is narrow and explicitly authorized by the owner: an exact item name on the client's own VENTAS document when that item name literally states an existing sales category.

2C. Exactly what made up the 1,733-row pre-Claude gold

The pre-Claude snapshot used by the recovery is:

`Data/gold/_master_gold.backup_20260811_104643.csv`

Its row math is exact:

Gold source tag	Rows	Where it came from	Who/what supplied the label	Trust interpretation
<code>client_product_rule</code>	577	Client product lists and dated product-list additions	Client-defined product-to-category mapping	Tier 1 direct client definition
<code>direct_client_example</code>	128	<code>xml_examples_list.xlsx</code> row examples	Client-confirmed example rows	Tier 1 direct client definition
<code>client_service_rule</code>	7	Client "servicios" sheet	Client-defined future service descriptions	Tier 1; forward-looking, not found in historical raw lines
<code>file_audit</code>	384	XMLs filed inside category-named example folders	Folder placement checked against client category descriptions	Tier 2; client-filed but the folders were known to be somewhat polluted

Gold source tag	Rows	Where it came from	Who/what supplied the label	Trust interpretation
<code>file_audit_corrected</code>	15	Misfiled/conflicting folder examples	Semantic re-audit corrected the folder label	Tier 2 correction, with reasons saved per row
<code>silver_audit</code>	622	AI-generated silver candidates promoted after an audit verdict	Mixed AI-assisted audit history described below	Tier 3; lower trust than direct client and file evidence
Total	1,733			

The first three direct-client streams total **712 rows**. The two folder-audit streams total **399 rows**. The silver-audit stream contributes **622 rows**. Therefore:

```
712 direct-client rows
+ 399 client-file/folder-audit rows
+ 622 audited-silver rows
= 1,733 original gold rows
```

Who audited the 622 silver rows?

This needs careful wording because the repository does not record a signed human reviewer identity on every row.

- The project rules say Ollama/Qwen only proposed labels and silver should enter gold only after manual audit.
- The saved row metadata explicitly mentions “Ollama + Claude” on 233 of the 622 rows.
- 136 rows mention a keyword-audit path.
- 21 rows still carry `verify_flag=client_check`, meaning a client confirmation was still desired.
- Other rows contain category-specific audit reasons, but not a reliable named-human signoff field.

Therefore this report calls them **AI-assisted audited silver**, not “fully human-certified gold.” They were already in the pre-Claude snapshot and were preserved except for three direction errors. Their mixed provenance is a remaining data-quality limitation.

2D. What the silver files actually contain

`Data/silver/` holds three different kinds of CSV, and adding all their row counts together would be meaningless.

Silver object	Rows	Meaning
<code>_candidate_pool.csv</code>	4,135	Raw candidate invoice lines selected for possible labeling; 4,129 COMPRAS and only 6 VENTAS
<code>_pool_classified.csv</code>	4,135	Ollama/Qwen suggestions for the same pool, including blanks/rejections
Per-category audit ledgers	1,672	Working audit rows routed into category files
Ledger rows with a nonblank verdict	1,076	Some audit decision exists
Verdicts marked REJECT	64	Explicitly not gold material
Non-rejected usable verdicts	1,012	Potential promotion material, still subject to conflicts/dedup/trust rules

The original gold contained 622 `silver_audit` rows, not all 1,012 usable ledger rows. Claude's broad re-promotion tried to recover more of the remaining audited ledger, which is where the scope and contradiction problems entered.

2E. Income data before the repair

Among the 1,733 original gold rows, only eight rows had an `ING-*` label:

Income class	Original rows	What the audit found
<code>ING-0.1</code> milk sales	1	Real sale, but too few to train SetFit
<code>ING-0.2</code> cow sales	3	Two were actually cow purchases (<code>SA-00757</code> , <code>SA-00758</code>)

Income class	Original rows	What the audit found
ING-0.3 heifer sales	1	Real, but too few to train
ING-0.4 calf sales	2	Trainable minimum, but extremely weak
ING-0.6 firewood sales	1	Actually a firewood purchase (SA-00874)
Total	8	Three direction errors; five remaining verified income rows

The milk deduplication bug happened in the path from audited silver to gold. All 47 milk invoice lines existed upstream, but the historical promotion key admitted only one milk row into gold. The trainer then removed that one-example class.

This pinpoints the failure:

```
raw milk rows: 47          GOOD - data existed
silver milk audit rows: 47  GOOD - proposed/audited as ING-0.1
old gold milk rows: 1      BROKEN - promotion dedup collapsed them
v1.1 trainable ING-0.1 class: absent  CONSEQUENCE - trainer dropped <2
frontend display: expense name shown  AMPLIFIER - review suggestion looked final
```

2F. What Claude added to the broad 2,262-row master

The saved broad Claude-era master is `Data/gold/_master_gold.csv`. Its exact source composition is:

Source	Rows in broad master	Relationship to pre-Claude gold
Direct client streams	712	Preserved
Folder audit streams	399	Preserved
Existing <code>silver_audit</code>	619	Three direction-error rows removed
New <code>silver_audit_v2</code>	469	Broadly re-promoted audited silver
<code>raw_ventas_harvest</code>	59	Verified non-milk sales rows added from raw VENTAS
<code>synthetic_floor</code>	4	One synthetic heifer plus three synthetic firewood examples
Total	2,262	

The arithmetic is:

```
1,733 pre-Claude gold
-   3 purchase-as-income rows
+ 469 broad silver promotions
+   59 raw VENTAS harvest rows
+    4 synthetic rows
= 2,262 broad Claude-era rows
```

Why only 59 raw sales rows in that version? Claude's broad silver promotion separately supplied 47 new milk rows, so its raw-harvest stage mainly added cow/heifer/calf rows. This split across two provenance streams made the final sales composition harder to reason about. It also left 48 milk rows in broad gold - the old one plus 47 promoted rows - even though the primary raw VENTAS corpus contains 47 milk lines.

The broad master was repaired back to zero cross-label contradictions, but “zero contradictions” does not prove all 469 new silver labels are correct. It only proves identical normalized inputs are not assigned different labels.

2G. What Codex accepted, rejected, and preserved

Codex did not clean the broad master in place. It created a separate candidate from the pre-Claude snapshot.

Decision	Rows	Reason
Preserve pre-Claude source rows initially	1,733	Known rollback anchor
Quarantine <code>SA-00757</code> , <code>SA-00758</code> , <code>SA-00874</code>	3	Source documents were COMPRAS but labels were income

Decision	Rows	Reason
Reject broad <code>silver_audit_v2</code> from focused candidate	469	Unrelated mixed-trust labels not needed for the milk repair
Reject Claude synthetic rows	4	No verified real firewood sale; no need to fabricate coverage
Add exact verified raw VENTAS rows not already present	105	Item name and outgoing direction explicitly identify five sales categories
Preserve Claude artifacts separately	all files	Historical evidence and rollback, not selected training truth

The focused arithmetic is:

```
1,733 pre-Claude gold
-   3 verified direction errors
+ 105 verified raw VENTAS inputs not already present
= 1,835 provenance rows
```

The 105 additions are:

Exact-sales class	Already represented in original gold after quarantine	New raw VENTAS rows added	Final provenance count
Milk <code>ING-0.1</code>	1	46	47
Cows <code>ING-0.2</code>	1	14	15
Heifers <code>ING-0.3</code>	1	1	2
Calves <code>ING-0.4</code>	2	44	46
Total	5	105	110 verified income provenance rows

The raw-sales audit also found that 13 of the 118 canonical raw VENTAS model inputs were already represented in the original data, so only 105 needed to be appended. The final validator counts all 118 canonical raw sales lines as covered by exact lookup; the focused gold contains 110 income provenance rows because some raw invoice lines collapse to an already represented model input.

2H. Why 1,835 rows become 1,580 model inputs

Gold keeps invoice/example rows for traceability. Training should not treat identical final text as new information.

In the focused candidate:

Source	Provenance rows	Distinct normalized inputs inside that source	Redundant rows inside the source
<code>client_product_rule</code>	577	577	0
<code>client_service_rule</code>	7	7	0
<code>direct_client_example</code>	128	80	48
<code>file_audit</code>	384	384	0
<code>file_audit_corrected</code>	15	15	0
<code>silver_audit</code> after quarantine	619	422	197
<code>raw_ventas_exact</code>	105	105	0

The distinct counts in that table sum to 1,590, but ten inputs overlap across different source streams. Therefore:

```
1,835 provenance rows
- 48 repeated model inputs inside direct client examples
- 197 repeated model inputs inside silver audit
- 10 net overlaps between different source streams
```

```
= 1,580 distinct normalized model inputs
```

No provenance row is erased from the candidate master. The training loader groups rows by normalized final model text and gives SetFit one representative per same-label group. If one normalized input has two different labels, training fails instead of choosing silently. The focused candidate has zero such cross-label groups.

Normalization includes lowercasing, accent removal, punctuation collapse, and the same numeric-description cleaning used by the model. This is why a “CSV row count” and a “learning-example count” are not interchangeable.

2I. How the 1,580 inputs are used

The deterministic split accounts for every distinct normalized input:

Final use	Inputs	Explanation
Regular training	1,247	Classes large enough to reserve validation rows
Weak/train-only	21	Real classes with 2-4 distinct inputs; trained but not honestly measurable with a separate slice
Validation	311	Held out from candidate training
Excluded	1	ADM-1.9, only one distinct input
Total	1,580	Every input has one explicit status

Regular plus weak training is 1,268 inputs. There is zero normalized input overlap between the 1,268 training inputs and 311 validation inputs.

2J. The safest mental model for the data

Use these questions in order whenever someone gives you a count:

- 1 Is this a **document count**, an **invoice-line count**, a **labeled provenance-row count**, or a **distinct model-input count**?
- 2 Which source stream supplied the row?
- 3 Who or what supplied the category label?
- 4 Was it direct client evidence, folder evidence, AI-assisted silver, exact raw-sales evidence, or synthetic?
- 5 Was the row quarantined, collapsed as a same-label duplicate, trained, validated, or excluded?
- 6 Which transformation version produced the count?

A count without those six answers is not enough to judge data quality.

3. What SetFit is, without the jargon

The selected model is SetFit using the multilingual MPNet sentence encoder.

It has two major parts:

- 1 **Embedding body**: turns invoice text into a list of 768 numbers that represent meaning.
- 2 **Logistic Regression head**: learns which region of that number-space belongs to each accounting code.

The model input is built from:

```
item_text | cleaned description | provider
```

For example:

```
VENTA DE LECHE | 12500 LITROS TANQUE 2 | CLIENTE X
```

Numeric-only descriptions are removed. Training and inference must build this text the same way.

The most important SetFit fact in this incident

The model does **not** understand a category merely because the human-readable category name exists in a spreadsheet. The Logistic Regression head needs examples for that category. If **ING-0.1** has no trainable examples, there is no output neuron/row for **ING-0.1**, and the model literally cannot choose it.

Renaming the label to “Milk Sale” would not solve that. Category names are display metadata; they are not semantic prompts to this classifier.

4. What “prediction,” “decision,” and “final code” mean

These are three different ideas:

- **Prediction:** the model's best suggestion.
- **Decision:** whether the suggestion is safe enough to accept automatically.
- **Final code:** the result that should actually be treated as accepted.

In the original incident, the model produced a weak expense suggestion, but the backend marked it **review_required** and left **final_code** empty. The frontend showed the weak suggestion like a final answer. That display mistake amplified a model limitation into a client-facing trust failure.

The backend did express uncertainty. The interface hid it.

PART II - HOW THE INCIDENT STARTED

5. The visible symptom

All 125 sales lines were shown as expenses. The most embarrassing example was milk revenue displayed as road-maintenance expense.

Observed old behavior included:

Raw sales phrase	Count	Old wrong behavior
VENTA DE LECHE	47	Mostly EXP-14.1 Mantencion Caminos; some freight
VENTA DE VACAS	18	Mostly road-maintenance expense
VENTAS TERNEROS	50	Mostly another expense code
VENTA DE VAQUILLAS	2	Expense
VENTA DE TERNERAS	1	Expense
Other/asset sales phrases	7	No verified category in current taxonomy

This was not a one-row typo. It exposed a broken path from data promotion through model training and user display.

6. The complete root-cause chain

Step 1 - the milk rows existed and had been audited

There were 47 raw VENTA DE LECHE lines. The silver audit ledger contained all 47 and labeled them ING-0.1 with high audit confidence.

Step 2 - the promotion deduplication key was too narrow

The historical promotion script treated rows as duplicates using approximately:

```
normalized item name + category code
```

It ignored description and provider, even though those fields are part of the model input. Because every milk row shared the item name and category, 47 genuine training texts collapsed to one.

Step 3 - SetFit training silently excluded the one-example class

Contrastive SetFit training needs at least two examples in a class to form a positive pair. The trainer excluded classes with fewer than two examples, logged the exclusion, and continued.

That behavior is understandable in isolation, but dangerous without a release gate. The class representing the client's core milk revenue disappeared from the model.

Step 4 - the model was forced to guess among the wrong classes

The v1.1 classifier had 66 classes. ING-0.1 was absent. A classifier must distribute probability among the classes it has, so it chose the nearest expense classes with low confidence.

It was not “confidently convinced milk is road maintenance.” It was saying: “none of my available answers fits well, but I must still rank something.”

Step 5 - evaluation failed to protect the business-critical case

The old validation split had no income examples and included leakage from duplicate texts. Its aggregate accuracy of 0.7441 did not measure the milk use case. Five trained classes also had no validation support.

Step 6 - two income classes were poisoned or too weak

Two of the three old ING-0.2 examples were actually cow purchases from COMPRAS invoices. The training schema lacked direction, so a purchase could be presented as income. Other income classes had only two or three examples.

Step 7 - the interface ignored the review contract

The backend returned low confidence, `review_required`, and no final code. The frontend surfaced the preliminary name as if accepted and did not make uncertainty obvious.

7. What was ruled out

The incident was reproduced locally with the original model, so it was not caused by:

- ONNX quantization;
- Cloud Run;
- GCP;
- a network service;
- Ollama failing to label the milk rows;
- a database constraint;
- the confidence threshold itself.

The threshold actually did its job by refusing the weak answer. The missing class and the display contract were the important failures.

PART III - WHAT CLAUDE DID

8. A fair summary before the details

Claude made several valuable discoveries: it found the original deduplication cause, identified purchase-as-income rows, found contradiction and leakage bugs, preserved backups, and proposed the right high-level architecture of exact rules plus ML for the long tail.

The problem was execution control. A small, high-confidence milk repair expanded into broad silver promotion, synthetic classes, several candidate models, business-rule API changes, replay scripts, database-contract changes, and Mac memory experiments. Some changes were useful, some were incomplete, and several claims were made before the end-to-end path had been verified.

The following chronology is reconstructed from Claude's pasted transcript, saved handover documents, generated candidates, and the Git snapshot at [76a3f76/78d44f2](#).

9. Claude phase A - broad data promotion

Instead of limiting the first repair to verified sales rows, Claude attempted to recover and promote a much larger set of audited silver examples. The broad candidate eventually reached 2,262 gold rows, compared with the 1,733-row pre-session gold.

Why that was risky:

- much of silver had only bulk-review provenance;
- unrelated classes were changed while the urgent bug concerned obvious sales;
- the promotion contained conflicting labels;
- stale generated category views no longer matched the master;
- the resulting model mixed the milk fix with hundreds of unrelated decisions.

This made it difficult to answer a simple question: "Did the milk fix help without harming the existing model?"

10. Claude phase B - a contradiction bug introduced by the fix

Claude initially changed deduplication to include item, description, provider, and category code. That sounded better than the original narrow key, but it created a new logical hole.

If the category code is part of the duplicate key, these two records appear different:

```
same model input -> EXP-14.1
same model input -> ING-0.1
```

The label itself makes the keys different, so both pass. The model is then shown the same input with two "correct" answers.

Measured effect:

- contradictions before the session: 0 groups;
- contradictions after the first promotion: 46 normalized keys;
- affected rows: 96;
- later rejected disagreements: 53 incoming silver rows.

Claude caught and admitted this bug. It changed the policy so established gold wins and conflicting promotions are logged instead of appended. That correction was useful, but it also demonstrated why the broad rewrite needed an independent audit.

11. Claude phase C - raw VENTAS harvest and synthetic income

Claude correctly inspected the raw VENTAS lines and found the real phrase distribution:

Phrase	Raw count	Verified category status
VENTA DE LECHE	47	Verified ING-0.1
VENTA DE VACAS	18	Verified ING-0.2 , but three old purchase-labeled rows needed quarantine
VENTA DE VAQUILLAS	2	Verified ING-0.3
VENTAS TERNEROS	50	Verified ING-0.4
VENTA DE TERNERAS	1	Verified ING-0.4
Asset/other phrases	7	Taxonomy answer not verified

It also found three direction errors:

- **SA-00757** - cow purchase labeled as cow-sale income;
- **SA-00758** - pregnant-cow purchase labeled as cow-sale income;
- **SA-00874** - firewood purchase labeled as firewood-sale income.

However, Claude added three synthetic firewood-sale examples to make **ING-0.6** trainable even though there was no verified firewood sale in the 125 raw VENTAS rows. This transformed “we have no real evidence” into a trained class. The recovery rejected that choice.

12. Claude phase D - discovery of train/validation leakage

Claude asked three subagents for audits. All three failed before completing: two hit account/session limits and one hit a network error. One partial audit raised a leakage concern, which Claude then investigated directly.

The key finding was real: multiple provenance rows produced identical final model text. Copies landed in both training and validation, so the model was tested on text it had already seen.

Historical measurements varied because the gold and split were changing during the session:

- the transcript measured 56 of 448 validation rows leaking;
- the later independent audit measured 57 of 340 v1.1 validation rows, representing 30 distinct leaked texts.

These figures are not contradictory; they refer to different dataset/split snapshots. The stable conclusion is that exact-input leakage existed and inflated validation results.

Claude correctly changed model loading to collapse same-label duplicate model inputs. It also correctly stopped describing the “20/20 income” result as unseen-wording generalization.

13. Claude phase E - deterministic rules and direction masking

Claude proposed an important architectural improvement:

- exact known sales phrases should not depend on a probabilistic model;
- COMPRAS should never resolve to an income code;
- asset sales with no valid taxonomy category should be reviewed rather than forced into the nearest class.

This direction was good. The first implementation was not end-to-end safe.

The independent audit found:

- 1 **source="business_rule"** was not allowed by the API response schema, so the exact milk request returned a local HTTP 500.
- 2 The production loader parsed transaction direction but dropped it before sending the inference request.

- 3 The database SQL enums did not allow `business_rule`.
- 4 The “review with no prediction” response conflicted with loader and database non-null assumptions.
- 5 The assign rule still involved model behavior in ways that could veto or complicate an authoritative fact.
- 6 The local 12,071-row replay did not call the business rules or direction mask at all.
- 7 The test suite had only six passing tests and none covered the new rule path.

Therefore “the fix works end to end” was premature. A direct in-process predictor experiment worked, but the actual application chain did not.

14. Claude phase F - the Mac memory detour

Full SetFit training repeatedly failed on a 16 GiB Mac using Apple's MPS accelerator. Claude moved through several explanations and workarounds:

- 1 It first suspected other applications consuming memory.
- 2 It then suspected long uptime and swap; the user rebooted, but the same error returned.
- 3 It identified a repeated 732.43 MiB allocation matching the token-embedding gradient size.
- 4 It lowered batch size from 8 to 4; this did not solve the failure.
- 5 It added gradient checkpointing; memory improved slightly but still failed.
- 6 It experimented with cache clearing.
- 7 It considered disabling the MPS high-watermark safety limit.
- 8 It froze the token-embedding table, which completed faster and under the memory ceiling.

Claude then argued frozen embeddings were a defensible engineering configuration and statistically similar. The user objected for a valid reason: with limited training data, the chosen SetFit approach was supposed to adapt the embedding space, and memory constraints should not silently change the modeling strategy.

The frozen run was a useful diagnostic, but it was not an acceptable final answer to the user's requirement.

15. Claude phase G - candidate models and misleading comparisons

Several additive candidate directories were created and left local:

- `models/setfit_base_v1_2_0/`
- `models/setfit_base_v1_2_0_clean/`
- `models/setfit_base_A/`
- `models/setfit_base_v120/`
- `models/_checkpoints_base/`

The latest Claude candidate `setfit_base_v120` reported:

Measure	Value
Gold rows	2,262
Distinct normalized inputs	2,009
Train / validation	1,613 / 395
Accuracy	0.6329
Macro-F1	0.5940
Top-3	0.8025
Income slice	20/20

Measure	Value
Trained classes with no validation support	8

The 20/20 result used distinct full invoice texts but shared canonical item phrases with training. It was useful evidence of repeated-phrase recognition, not unseen-language generalization.

A 12,071-row local replay showed large behavior changes, including 120 of 125 sales predicted as income. But the rows did not all have trusted labels, and the replay omitted the new rules and direction mask. It measured changed behavior, not accuracy of the proposed architecture.

16. Claude phase H - dangerous export path

The inherited exporter accepted a variant name and derived a model directory such as `models/setfit_base`. It could not safely select tagged candidates such as `models/setfit_base_v120`.

That meant a command labeled “v1.2.0” could accidentally export the old v1.1 model under the new version name. Worse, the exporter recursively removed an existing artifact directory before parity validation.

The recovery correctly stopped before export. No ONNX bundle was generated.

17. What Claude got right and wrong

Area	Useful work	Failure or limitation
Root cause	Found the milk dedup/class-exclusion chain	Expanded scope before locking a minimal repair
Data	Found three purchase-as-income rows and raw sales counts	Broad promotion mixed in unrelated silver; added synthetic firewood
Integrity	Found contradictions and leakage	Introduced the first contradiction bug; reports changed while audits ran
Architecture	Proposed exact rules and transaction direction	API, loader, persistence, replay, and tests were not aligned
Evaluation	Corrected the unseen-wording claim	Compared changing datasets and reported aggregate behavior too early
Memory	Gathered useful allocation evidence	Misidentified the root cause and recommended frozen embeddings/unbounded ceiling
Safety	Preserved backups and additive models	Exporter remained capable of wrong-model packaging and destructive overwrite
Documentation	Created many Field Guide files	Historical plans, corrections, and current truth became mixed together

The lesson is not “Claude did nothing useful.” The lesson is that good discoveries were interleaved with unsafe scope growth and unverified integration claims.

PART IV - WHAT CODEX DID

18. Step zero - preserve and make rollback real

Before changing the recovery, Codex saved the visible state into local Git.

Commit	Purpose
a60e43f	Original main baseline: packaged v1.1.0
76a3f76	Snapshot of visible Claude code/docs/handover state
78d44f2	Snapshot of previously ignored training sources and data
d3e2360	Independent audit and safe repair plan
9776523	Validated focused recovery and fully trained model

Branches:

- baseline audit branch: [codex/pre-fix-audit-20260811](#) at [d3e2360](#);
- recovery branch: [codex/milk-fix-clean](#) at [9776523](#).

Protected originals:

- [models/setfit_base/](#);
- [artifacts/v1.1.0/](#);
- [Data/gold/_master_gold.backup_20260811_104643.csv](#).

No reset, destructive cleanup, remote push, deployment, API deployment test, Supabase write, or online release action was performed.

19. Independent audit findings

Codex did not accept the handover's "works end to end" statement. It traced the actual application path and recorded ten critical findings:

- 1 Canonical milk request failed the local API response contract.
- 2 The real loader did not send transaction direction.
- 3 Persistence could not represent the new source/review shape.
- 4 The full-data replay did not execute the rules being claimed.
- 5 The latest Claude model was weaker and less validated than described.
- 6 "Exact model input" dedup terminology did not precisely match normalized grouping.
- 7 Synthetic [ING-0.6](#) had no real sale evidence.
- 8 generated gold views and reports were stale.
- 9 the exporter could package the wrong model and delete an existing artifact.
- 10 direction safety was applied too narrowly and too late.

Audit verdict at that point: do not deploy, export, push, or write results to Supabase.

20. The focused-data decision

Codex rejected the 2,262-row broad Claude corpus as the recovery basis because the extra unrelated promotions could not be audited quickly enough.

The focused candidate policy was:

```

pre-Claude gold
- three verified purchase-as-income errors
+ verified exact raw VENTAS rows
+ no unrelated broad silver
+ no synthetic income

```

Measured result:

Data operation	Rows
Start from pre-Claude gold	1,733
Quarantine verified direction errors	-3
Add distinct verified raw VENTAS rows not already present	+105
Raw verified sales already present	13
Final provenance rows	1,835
Distinct normalized model inputs	1,580
Synthetic rows	0
Cross-label contradictions	0

Income coverage in the focused candidate:

Class	Inputs	Treatment
ING-0.1 milk	47	Trained and validation-supported; exact lookup protected
ING-0.2 cows	15	Trained and validation-supported; exact lookup protected
ING-0.3 heifers	2	Weak/train-only; exact lookup protected
ING-0.4 calves	46	Trained and validation-supported; exact lookup protected
ING-0.5 other animals	0 verified sales	Not invented
ING-0.6 firewood	0 verified sales	Synthetic Claude examples removed

21. The split and leakage repair

The final split is deterministic and saved in [Data/candidates/recovery_v1_2_0/split_seed42.csv](#).

Split	Inputs	Meaning
Train	1,247	Normal classes with validation support
Train weak	21	Real but too small for a reliable validation slice
Validation	311	Held out from candidate training
Excluded	1	ADM-1.9, only one distinct input

Together, regular and weak training total 1,268 inputs. Normalized train/validation overlap is zero.

The recovery keeps weak classes explicit rather than silently hiding them. **ING-0.3** is one of the weak classes; its exact phrase is therefore guaranteed by a deterministic lookup, not by pretending two examples establish broad ML quality.

22. Exact sales lookup - the product-style solution the user allowed

The repository already used exact deterministic lookups for known products and electricity meters. The recovery adds the same idea for five verified sales phrases.

Transaction type	Exact normalized item	Code
VENTAS	VENTA DE LECHE	ING-0.1
VENTAS	VENTA DE VACAS	ING-0.2
VENTAS	VENTA DE VAQUILLAS	ING-0.3
VENTAS	VENTAS TERNEROS	ING-0.4
VENTAS	VENTA DE TERNERAS	ING-0.4

Safety properties:

- The match key includes `transaction_type=VENTAS`.
- The same words under COMPRAS do not trigger a sale lookup.
- A verified match returns immediately with score 1.0 and does not ask the model to vote.
- Invalid transaction types are rejected by the request schema.
- All `ING-*` probabilities are removed from COMPRAS model rankings.
- Unknown VENTAS phrases may get a suggestion, but can never auto-accept.
- Unverified firewood and asset-disposal mappings were removed.

There are 118 verified canonical VENTAS lines protected by these five rules and seven unknown sales lines that are forced to human review:

Unknown phrase	Count
VENTA CAMIONETA	3
VENTA DE ACTIVO FIJO	2
OTROS INGRESOS	1
maquinaria	1

Review is correct here because the current taxonomy does not contain a safely verified answer for these phrases.

23. Repairing the entire local route

The recovery aligned the previously mismatched layers:

- API response source now allows `business_rule`.
- Request transaction type is constrained to `COMPRAS` or `VENTAS`.
- The batch/loader payload now includes transaction type.
- SQL source constraints include `business_rule`.
- Exact assign rules short-circuit the model.
- Unknown sales return a normal non-empty model suggestion but remain review-only, avoiding the broken empty-prediction persistence shape.
- New tests cover exact rules, wrong direction, unknown sales, masks, schema behavior, and loader payload behavior.

These are local contract tests. No deployed endpoint was called because no recovery service is deployed.

24. The real Mac MPS memory diagnosis

The user rejected frozen embeddings, so Codex treated full encoder training as a requirement and ran controlled memory experiments.

Experiment 1 - change optimizer, keep dynamic padding

Adafactor reduced optimizer-state pressure, but memory still grew:

Step	Live tensor memory	MPS driver memory
1	about 1.05 GiB	5.05 GiB
12	about 1.05 GiB	14.41 GiB

The live tensors stayed nearly flat while driver memory grew. That evidence contradicted the theory that AdamW state or the token-gradient alone caused the growth.

Experiment 2 - fixed 64-token padding

The contrastive collator had been dynamically padding each batch to its own longest sequence. On MPS, each new tensor shape can cause a new compiled graph/allocation pattern to be cached.

Padding every batch to the already chosen maximum of 64 tokens gave MPS one stable shape to reuse.

Run	Padding	Optimizer / batch	Driver-memory result
Failing smoke	Dynamic	Adafactor / 4	5.05 to 14.41 GiB in 12 steps
Repaired smoke	Fixed 64	Adafactor / 4	Plateau around 4.42 GiB through 20 steps
Release configuration	Fixed 64	AdamW / 8	Plateau around 6.60 GiB through 20 steps

The key fix was fixed batch shape, not frozen embeddings and not an unbounded memory safety override.

25. Full SetFit training configuration

The recovery trainer adds fail-closed assertions:

- all 278,043,648 encoder parameters must be trainable;
- the tracked token embedding table must change;
- `PYTORCH_MPS_HIGH_WATERMARK_RATIO=0.0` is refused;
- fixed-length padding must be active;
- the split and gold hashes are saved;
- weak and excluded classes are explicit.

Final run:

Setting	Value
Base encoder	<code>paraphrase-multilingual-mpnet-base-v2</code>
Device	Apple MPS
Embedding training budget	1,500 steps
Batch size	8
Optimizer	AdamW
Gradient checkpointing	On
Padding	Fixed to 64 tokens
Full encoder trainable	Yes, 278,043,648 / 278,043,648 parameters
Token embedding max absolute delta	0.0023982
Duration	22.46 minutes

Setting	Value
Peak recorded MPS driver memory	6.6092 GiB
Unbounded watermark	No

This directly answers the user's objection: the release candidate does not freeze embeddings.

26. Honest model evaluation

Candidate's own locked validation split

On 311 held-out candidate inputs:

Metric	Result	Beginner interpretation
Accuracy	0.7524	Top answer correct on about 75%
Macro-F1	0.7316	Average class quality, giving small classes a voice
Top-3 accuracy	0.8746	Correct answer in first three on about 87%
Canonical income slice	21/21	All held-out known-phrase income rows top-1 correct

The 21 income validation rows are distinct full model inputs, but their canonical item phrases also occur in training. This is an honest repeated-phrase test, not evidence of arbitrary unseen income wording.

The first baseline comparison was rejected

The first comparison applied v1.1 to the candidate validation split and appeared to show v1.1 accuracy 0.8167 versus recovery 0.7524. That was not a fair test because many candidate validation inputs had been part of v1.1 training.

Codex preserved that failed diagnostic in `model_comparison.json` instead of deleting it, then refused to use it as the release conclusion.

Fair shared-blind comparison

To compare the models on inputs unseen by both:

- 1 Start with 340 v1.1 validation rows.
- 2 Remove 29 duplicates inside that validation set.
- 3 Remove 32 inputs that were not unique in original gold and could have leaked into v1.1 training.
- 4 Remove 221 inputs present in candidate training.
- 5 Evaluate the remaining 58 rows.

Results:

Metric	v1.1	Recovery	Delta
Accuracy	0.6034	0.7069	+0.1035
Macro-F1	0.5886	0.6323	+0.0437
Top-3 accuracy	0.8103	0.8621	+0.0518

The recovery wins all three metrics on this shared-blind regression gate. However, 58 is a small set and it contains no income rows. It is evidence against a broad regression, not a complete benchmark of every category.

27. Final local verification

The saved final validator reports `status: pass` and checks:

- focused gold counts and hashes;

- zero synthetic examples;
- zero cross-label contradictions;
- exact split counts;
- exact lookup coverage: 118 verified and 7 review-only;
- full encoder trainability;
- token embeddings changed;
- fixed padding enabled;
- unbounded MPS watermark disabled;
- fair shared-blind improvement;
- candidate locked accuracy and income gates.

The complete local test suite passed 15 of 15 tests at the recovery commit.

Again: this verifies the local source/model candidate. It does not verify an ONNX export, a cloud deployment, the frontend, or a remote API because those actions were deliberately not performed.

PART V - THE CURRENT ARCHITECTURE

28. The classification cascade now

```
invoice line
|
|-- known electricity meter? ----- yes --> meter lookup, auto-accept
|
|-- verified exact VENTAS phrase? ---- yes --> business rule, auto-accept
|
|-- exact product lookup? ----- yes --> product mapping plus conflict checks
|
|-- otherwise -----> SetFit encoder + LR head
|
|   |-- COMPRAS: remove all ING-* choices
|   |-- unknown VENTAS: suggestion only, force review
|   |-- weak/low-confidence/small-margin: review
|   |-- otherwise: auto-accept
```

This is a hybrid system:

- deterministic knowledge for exact facts;
- machine learning for variable, long-tail language;
- explicit review when the taxonomy or evidence is insufficient.

That architecture is stronger than forcing one model to solve all three problems.

29. Why exact lookup is not “cheating”

If **VENTA DE LECHE** in a VENTAS invoice has a stable client-approved answer, asking a probabilistic model every time adds avoidable failure. Software systems routinely use a lookup for stable identifiers and ML for ambiguous inputs.

The existing product and meter lookups already followed this principle. The sales lookup extends the same pattern with transaction direction in the key.

The model still learns the income classes. The lookup is a safety guarantee for the exact repeated business phrases.

30. Why transaction direction must travel end to end

LECHE can appear in different business contexts. “Sell milk” and “buy milk” must not be treated identically. Direction is not cosmetic metadata; it changes which labels are possible.

The chain must preserve direction from source folder to parsed row, API request, predictor, decision, and storage. Dropping it at any boundary silently disables the safety layer.

The current request schema still allows omission for backward compatibility. That means callers outside the repaired loader can disable the protection by omitting direction. Before production release, missing direction should be reviewed as a fail-closed contract decision.

31. What remains old or unbuilt

- The validated recovery model is PyTorch SetFit under `models/setfit_base_recovery_v1_2_0/`.
- `artifacts/v1.1.0/` remains the old deployed-form local artifact.
- There is no `artifacts/v1.2.0/` recovery export.
- There is no deployed recovery service.
- The frontend display behavior is unchanged by this work.
- The broad Claude candidate gold and models remain as historical/quarantined evidence, not the selected release candidate.

PART VI - FLAWS, AUDITS, AND OPEN RISKS

32. Data risks that remain

Seven unknown VENTAS rows

The current taxonomy does not have a verified answer for vehicle sales, fixed-asset sales, other income, and machinery. They are safely review-only, but a human accounting decision is still needed.

Very small classes

- ADM-1.9 has one distinct input and is excluded.
- ING-0.3 has two inputs and is train-only/weak.
- several expense/administration classes also have tiny validation support.

Exact lookup protects the known heifer phrase, but the ML model cannot be claimed broadly strong for tiny classes.

Unadjudicated Claude-era conflicts

The 53 rejected silver conflicts remain useful as a human review queue. They were not silently resolved or added.

Unlabeled long tail

Thousands of line items remain outside trusted gold. The recovery intentionally did not use “more data” as an excuse to accept lower-trust labels.

33. Model-evaluation risks that remain

- There is no large independent test set created before model/rule decisions.
- The 58-row shared-blind set is small and contains no income.
- Known sales validation shares canonical phrases with training.
- Accuracy alone does not measure accepted-prediction safety.
- Per-class support is small for multiple categories.
- Confidence calibration and auto-accept accuracy should be evaluated on a locked, human-labeled test set before deployment.

34. Engineering risks that remain

- The old exporter can select the wrong model and overwrite/delete an artifact path before validation.
- PyTorch candidate behavior has not been compared with a new ONNX export.
- SQL migration was edited locally but not applied remotely.
- The optional transaction type still permits unprotected legacy calls.
- Frontend presentation of preliminary predictions versus final decisions remains outside this branch.
- Stale historical documents can mislead future agents unless they read the truth map below.

35. Why “15/15 tests” does not mean “ready to deploy”

Tests answer the questions they contain. The new tests show that the local rule and contract logic behaves as designed. They do not prove:

- cloud startup;
- ONNX parity;
- database migration safety;

- frontend display;
- every category's business correctness;
- production latency or memory;
- accepted-prediction accuracy on future client data.

Passing tests are necessary evidence, not universal proof.

PART VII - ROLLBACK AND RECOVERY MAP

36. How to go back safely

The selected recovery is additive. The original baseline is still present.

Return to the independent-audit point

Switch locally to branch `codex/pre-fix-audit-20260811` at commit `d3e2360`.

Return to the original main baseline

Switch locally to `main` at `a60e43f`.

Restore original gold

Use `Data/gold/_master_gold.backup_20260811_104643.csv` as the pre-Claude reference. Do not overwrite it.

Use the old model/artifact

- old local SetFit: `models/setfit_base/`;
- old deployment-form artifact: `artifacts/v1.1.0/`.

Recovery files are separate

- candidate data: `Data/candidates/recovery_v1_2_0/`;
- candidate model: `models/setfit_base_recovery_v1_2_0/`;
- reports: `reports/recovery_v1_2_0/`.

Large model files are tracked using Git LFS, so the recovery commit records them without replacing baseline model files.

37. What rollback cannot do

Git can restore local files and code. It cannot automatically undo a remote database migration or production write. That is one reason the recovery deliberately made no remote changes.

At the time of this report, there is no new remote state to undo.

PART VIII - APPLYING THE 15 FIELD-GUIDE HABITS

38. Project scorecard

Field Guide habit	What happened here	Current correction
1. Handover file	Claude created detailed handovers, but later truth diverged	This report and <code>RECOVERY_V1_2_0.md</code> state final local truth
2. Decisions.md	Useful decisions were recorded	Superseded frozen-embedding choice is preserved and explicitly corrected
3. Explicit comments	Business-rule intent was commented	Current rule/mask code explains why, not only what
4. Flow.md	Historical flow was documented	This report shows the repaired cascade and remaining old artifact boundary
5. Bug/Feature trace	BUG-001 recorded the incident	Historical "Fix" section is not final; this report completes the trace
6. Architecture.md	System shape was documented	Exact lookup + model hybrid is explained here in current form
7. Constraints.md	Local-only/no-push constraints existed	Codex followed them; no remote action occurred
8. Test checklist	Checklists existed	Claude claimed success before coverage; recovery added executable gates/tests
9. Rollback.md	Backups existed	Git snapshots, branches, hashes, separate candidate paths, and LFS make rollback concrete
10. Read the diff	Claude summaries hid integration gaps	Independent audit traced actual code/API/loader/SQL/replay paths
11. Ask why before what	Work broadened before a minimal causal fix was locked	Recovery chose a focused data policy first
12. Small requests only	Milk bug expanded into broad promotion, rules, DB, replay, and memory work	Recovery separated data, routing, memory, training, evaluation, and export gates
13. Session handoff	Handovers were created	Corrections and evidence status must be part of every future handoff
14. Version-pin context	"AI did this" was too vague	Git commits identify Claude snapshot, audit, and Codex recovery
15. Own the mental model	User could not know which claims were safe	This beginner report explains the causal chain, architecture, limits, and rollback

39. The deeper Field Guide lesson

Having documentation files is not enough. This project had many documents, yet they contained multiple time layers:

- an original plan;
- a plan revised after a bug;
- a correction after leakage was found;
- a memory workaround later rejected;
- an independent audit;
- a focused final recovery.

The solution is not "more documents" alone. Every document needs status, evidence, version, and a clear statement of whether it is historical, superseded, or current.

PART IX - DOCUMENT TRUTH MAP

40. What to trust first

Priority	File	Status	How to use it
1	docs/ML_MODEL_INCIDENT_RECOVERY_FIELD_REPORT.md	Current comprehensive narrative	Read first for full context
2	docs/RECOVERY_V1_2_0.md	Current concise technical truth	Use for candidate scope, metrics, limits, rollback
3	reports/recovery_v1_2_0/final_validation.json	Machine-readable final checks	Use for release-gate evidence
4	models/setfit_base_recovery_v1_2_0/run_manifest.json	Machine-readable training record	Use for parameters, memory, split, metrics
5	Data/candidates/recovery_v1_2_0/manifest.json	Machine-readable data lineage	Use for counts and hashes
6	reports/recovery_v1_2_0/model_comparison_fair.json	Fair comparison evidence	Use for shared-blind results
7	docs/INDEPENDENT_AUDIT_2026-08-11.md	Historical audit at pre-recovery point	Understand what was broken before 9776523
8	docs/DECISIONS.md	Append-only decision history	Read superseding entries, especially full embeddings
9	docs/BUG-001-milk-sales-misclassification.md	Root cause good; fix section historical	Do not treat synthetic-data plan as final
10	docs/RESULTS-v1.2.0.md	Claude-era, partly stale/confounded	Historical evidence only
11	docs/SESSION_HANDOVER_PROMPT.md	Claude-era handover	Historical reconstruction only
12	docs/HANDOVER.md	Older state snapshot	Historical reconstruction only

41. Historical artifacts that remain on purpose

Claude candidate models, broad gold, failed comparison reports, and stale handovers were not deleted. They are evidence of what happened and may help future analysis.

They are not the selected candidate.

The principle is quarantine, label, and preserve - not silently erase.

PART X - WHAT YOU CAN SAFELY SAY

42. Beginner-friendly status statement

You can say:

The milk-sale problem was not that the model randomly learned “road maintenance.” A data deduplication bug reduced 47 valid milk examples to one, and the training code then removed the one-example class. The old model had no milk-sale answer available. We rebuilt a focused local dataset from the pre-change backup, added only verified sales, removed three purchases mislabeled as income, added transaction-aware exact lookups for five canonical sales phrases, and fully retrained SetFit after fixing an Apple MPS dynamic-padding memory issue. The candidate is validated locally and reversible, but it is not exported or deployed yet.

43. Claims you should not make

Do not say:

- “The model is 100% accurate on sales.”
- “It generalizes to any new income wording.”
- “v1.2 is deployed.”
- “The API is production-tested.”
- “The frontend is fixed.”
- “All 125 sales have a known accounting category.”
- “The whole 2,262-row Claude dataset is trusted.”
- “Frozen embeddings were necessary on the Mac.”
- “0.7441 old accuracy was a clean benchmark.”

44. Current release decision

Local candidate status: technically validated for the focused recovery scope.

Deployment status: not ready to deploy as a complete system because safe export/ONNX parity, database migration, production contract, and frontend behavior remain separate gates.

Data status: focused candidate is the trusted recovery set; broad Claude additions remain quarantined/historical.

Model status: full SetFit encoder trained successfully; no frozen embeddings. **Known sales status:** 118 canonical rows protected by exact transaction lookup; seven unknown sales remain review-only.

PART XI - RECOMMENDED NEXT WORK

45. Before any deployment

- 1 Build a fail-closed exporter that requires an explicit model directory and refuses overwrite.
- 2 Export to a new temporary artifact path and run PyTorch-versus-ONNX parity checks before naming it v1.2.0.
- 3 Decide whether missing transaction type should be rejected or forced to review for every production caller.
- 4 Validate database schema migration in a disposable/local environment before any remote database change.
- 5 Fix the frontend to display `final_code`/decision correctly and clearly show review status and confidence.
- 6 Create a locked, human-labeled test set that is not used for training, rule authoring, threshold selection, or promotion.
- 7 Measure accepted-prediction accuracy and coverage, not only raw top-1 accuracy.
- 8 Obtain accounting decisions for the seven unknown sales phrases.

46. Data work after the urgent fix

- 1 Human-adjudicate the 53 rejected Claude-era conflicts using the documented trust hierarchy.
- 2 Review weak classes with tiny unique support.
- 3 Add real examples for `ING-0.5` and `ING-0.6` only when verified client data exists.
- 4 Regenerate gold category views from whichever master is formally accepted.
- 5 Version every accepted data release with source hashes, transformation version, class counts, split hash, and reviewer.

47. Operational rule for future AI sessions

Every future session should begin by reading the current recovery summary and manifests, then state:

- exact branch and commit;
- exact files in scope;
- whether any remote action is allowed;
- the rollback point;
- the test/evidence gate for completion;
- which historical documents are not current truth.

No change should be called complete only because an AI summary says it worked.

APPENDIX A - CHRONOLOGICAL LEDGER

A1. Before the incident investigation

Stage	Event	Evidence
v1.1 preparation	Gold contained only one milk-sale row and weak/poisoned income coverage	Old gold backup; v1.1 labels/model
v1.1 training	One-example classes were excluded; income absent from validation; duplicate leakage existed	Training code/logs; v1.1 split reconstruction
v1.1 packaging	Main reached a60e43f	Git history
Client use	Sales suggestions appeared as expense names; uncertainty/final-code distinction was not shown	Incident notes and local prediction snapshot

A2. Claude-era investigation

Order	Event	Outcome
1	Root cause traced to milk dedup and silent exclusion	Correct
2	Broad audited-silver promotion attempted	Scope expanded and trust became mixed
3	Dedup key included category label	46 contradiction groups / 96 affected rows introduced
4	Claude diagnosed and rebuilt from backup	Contradictions returned to zero in broad master
5	Raw VENTAS harvested; three purchases quarantined	Useful verified findings
6	Synthetic firewood examples added	Rejected by final recovery
7	Multiple MPS training attempts	OOM persisted; frozen embeddings used as workaround
8	Three audit subagents launched	All incomplete due limits/network; leakage clue survived
9	Exact-input leakage measured and loader dedup added	Correct integrity repair
10	Business rules/direction mask implemented	High-level design good; application integration incomplete
11	Candidate models and replay produced	Useful behavior evidence, but comparisons/confidence claims confounded
12	Handover/docs written	Rich history, mixed current and stale truth

A3. Codex independent recovery

Git time	Commit/event	Outcome
13:10	76a3f76 Claude snapshot	Visible work preserved locally
13:16	78d44f2 data/training snapshot	Ignored sources and recovery evidence tracked
13:19	d3e2360 independent audit	Ten critical failures documented; release blocked
After audit	focused 1,835-row candidate built	No synthetic data; no contradictions
After audit	exact transaction lookup repaired	118 canonical sales protected; 7 forced review
After audit	MPS experiments	Dynamic padding identified; fixed 64 solved growth
After audit	full 1,500-step SetFit training	22.46 min; 6.6092 GiB; embeddings changed
After audit	honest/fair evaluations	Candidate locked metrics and 58-row shared-blind win
14:07	9776523 recovery commit	Model/data/code/tests/reports tracked locally

APPENDIX B - GLOSSARY

Term	Plain-language definition
Auto-accept	The system considers the result safe enough to use automatically
Business rule	A versioned exact fact applied before ML
Candidate	A proposed local data/model version, not production
Class	One accounting category code the model can output
Confidence	How strongly the classifier favors an answer; not the same as correctness
Contradiction	Identical model input assigned two different labels
Deduplication	Removing records treated as duplicates
Direction	Whether the document is a purchase (COMPRAS) or sale (VENTAS)
Embedding	Numeric representation of text meaning
Gold	Trusted labeled examples used for training
Leakage	Validation contains text seen during training, making results look better than they are
Logistic Regression head	Final classifier trained on SetFit embeddings
Macro-F1	Per-class quality averaged so large classes do not dominate
MPS	Apple's GPU acceleration backend used by PyTorch on the Mac
ONNX	Portable exported model format used by the service artifact
Provenance	Record of where a training example came from
Review-required	Suggestion is not safe to accept automatically
SetFit	Few-shot text-classification approach using a sentence encoder and a classifier head
Silver	Candidate labels that need stronger verification than accepted gold
Top-3 accuracy	Whether the correct label appears among the first three suggestions
Weak class	A class with too little distinct data for dependable validation

APPENDIX C - PRIMARY EVIDENCE INDEX

C1. Incident and historical work

- Claude transcript: external attachment `pasted-text.txt` supplied with this task.
- `docs/BUG-001-milk-sales-misclassification.md`
- `docs/SESSION_HANDOVER_PROMPT.md`
- `docs/HANDOVER.md`
- `docs/RESULTS-v1.2.0.md`
- `docs/DECISIONS.md`
- Git commit `76a3f76`
- Git commit `78d44f2`

C2. Independent audit

- `docs/INDEPENDENT_AUDIT_2026-08-11.md`
- Git commit `d3e2360`

C3. Final focused recovery

- `docs/RECOVERY_V1_2_0.md`
- `Data/candidates/recovery_v1_2_0/manifest.json`
- `Data/candidates/recovery_v1_2_0/split_seed42.csv`
- `models/setfit_base_recovery_v1_2_0/run_manifest.json`
- `reports/recovery_v1_2_0/memory_smoke.json`
- `reports/recovery_v1_2_0/memory_smoke_fixed_padding.json`
- `reports/recovery_v1_2_0/memory_smoke_fixed_padding_adamw_b8.json`
- `reports/recovery_v1_2_0/model_comparison.json`
- `reports/recovery_v1_2_0/model_comparison_fair.json`
- `reports/recovery_v1_2_0/final_validation.json`
- Git commit `9776523`

C4. Current code paths

- `scripts/59_build_focused_recovery_gold.py`
- `training/train_recovery_setfit.py`
- `scripts/61_evaluate_recovery_model.py`
- `scripts/62_validate_recovery_release.py`
- `app/data/business_rules.csv`
- `app/inference/business_rules.py`
- `app/inference/predictor.py`
- `app/api/schemas.py`
- `Temp_Inference/classify_raw_invoices_to_supabase.py`
- `tests/test_transaction_rules.py`

- `tests/test_api.py`

APPENDIX D - FINAL OWNER CHECKLIST

After reading this report, the owner should be able to answer:

- Why did 47 milk rows become one gold example?
- Why could v1.1 never output `ING-0.1`?
- Why was the low-confidence expense name visible anyway?
- Which Claude discoveries were valuable?
- Which Claude changes were unsafe or incomplete?
- Why was broad 2,262-row gold rejected for the focused recovery?
- Why are there exactly five sales rules and seven review-only sales rows?
- Why is exact lookup appropriate alongside SetFit?
- Why were frozen embeddings rejected?
- What actually caused MPS memory growth?
- What do 0.7524 accuracy and 0.7316 macro-F1 mean?
- Why was one comparison rejected as unfair?
- What remains untested and undeployed?
- Which branch/commit/file returns to the previous state?

If any answer is still unclear, use the evidence index to trace the claim rather than accepting another summary on trust.

Closing statement

The original milk failure is now understood and locally repaired through two complementary protections: the model finally has verified income examples and a fully fine-tuned embedding body, while exact transaction-aware lookup guarantees the five canonical repeated sales phrases. The data and implementation are preserved in a separate Git commit with hashes, tests, manifests, and rollback points.

The recovery is strong precisely because it stops at what the evidence proves. It does not manufacture missing categories, call repeated phrases unseen language, confuse behavior changes with accuracy, or call a local PyTorch candidate deployed. The next safe step is controlled export and system-level release work - only after the owner approves that new scope.